

항만 물류 디지털 트윈 시뮬레이터

(JavaFX 기반 항만 운영 시뮬레이션 시스템)

프로젝트 계획서

제출일: 2026. 05. 15

팀명	کم공 카르텔	
	이름	역할
팀구성	이원훈	하역장 맵 및 선박 이동 구현
	허은총	크레인 적재·하역 모션 구현 및 화물 창고 구현
	정승호	UI 구현 (화주 시점 입력 인터페이스)

목포해양대학교 컴퓨터공학과

목 차

1. 개요 및 배경
 2. 기존 사례
 3. 주요 기능
 4. 시스템 구성도
 5. 핵심 구현 기술
 6. 개발 환경 명세
 7. 사용 데이터
 8. 개발 일정
 9. 업무 분장
- 참고문헌

1. 개요 및 배경

1.1. 개요

본 작품은 항만에서 이루어지는 화물 적재>Loading) 및 하역(Unloading) 업무 전반을 JavaFX 기반 GUI로 시뮬레이션하는 '항만 물류 디지털 트윈 시뮬레이터'이다. Oracle OpenJDK 17 및 Gradle 빌드 시스템을 기반으로, 외부 라이브러리를 최소화하고 Java OOP 원리를 항만 도메인에 직접 적용한다. 이 프로그램은 아래와 같은 핵심 기능들을 제공한다.

- ① 하역장 맵 시각화 및 선박 이동 시뮬레이션 (JavaFX Canvas + AnimationTimer)
- ② 크레인 기반 컨테이너 적재·하역 모션 구현 (JavaFX Timeline / KeyFrame)
- ③ 화물 창고 관리 및 입출고 로직 구현
- ④ 화주(Shipper) 시점 화물 정보 입력 UI (컨테이너 수 · 목적국 · 화물명 · 식별번호 등)
- ⑤ 멀티스레드(ExecutorService) 기반 다수 선박 동시 운항 처리

1.2. 배경

가. 항만 물류 디지털화의 필요성

현대 항만은 선박·화물·부두·크레인·인력이 복잡하게 얽힌 실시간 시스템으로, 국가 수출입 물류의 핵심 거점이다. 화물 위치 파악의 어려움, 적재·하역 일정 관리의 비효율, 부두 배정 혼선 등 다양한 운영 문제가 현장에서 지속 발생하며, 이를 소프트웨어로 모델링·시뮬레이션하여 운영 효율을 검증할 수 있는 시스템에 대한 필요성이 높아지고 있다.

나. 기존 솔루션의 기술적 제약

- 특정 상용 플랫폼에 종속되어 교육·연구 목적의 자유로운 활용이 어렵고, 라이선스 비용이 높다.
- 외부 라이브러리 의존도가 높아 학습 목적에서 핵심 OOP 로직 파악이 어렵다.
- 선박·부두·화물·크레인을 통합한 JavaFX 기반 시각화 시뮬레이터가 부재하다.
- 화주(Shipper) 시점의 사용자 입력 인터페이스를 제공하는 솔루션이 없다.

다. 본 프로젝트의 제안

본 프로젝트는 기존 솔루션의 한계를 극복하기 위해, Oracle OpenJDK 17 + JavaFX + Gradle 환경에서 Java OOP의 핵심 원리를 항만 도메인에 완전 적용한 시뮬레이터를 제안한다. 주요 특징은 다음과 같다.

- 외부 라이브러리를 최소화하고 Java OOP(캡슐화·상속·다형성·인터페이스)를 직접 적용한다.
- JavaFX AnimationTimer와 Timeline을 활용해 선박 이동 및 크레인 모션을 실시간 시각화한다.

- ExecutorService 기반 멀티스레드로 다수 선박의 동시 운항 및 부두 배정을 처리한다.
- 화주(Shipper) 시점의 FXML 기반 UI를 통해 화물 정보를 입력하고 시뮬레이터와 실시간 연동한다.
- Gradle 빌드 시스템(group: com.example)으로 모듈화된 프로젝트 구조를 관리한다.

2. 기존 사례

2.1. 유형별 사례 분석

가. 상용 항만 시뮬레이션 도구

1) 대표 사례

SIMPORT, FLEXSIM, SimPy 기반 항만 시뮬레이터

2) 기능적 특징

고성능 그래픽 엔진과 실제 데이터 연동을 제공하며 부두 처리량, 대기 시간 등 다양한 메트릭을 지원한다.

3) 한계점

고가 라이선스, 폐쇄 구조로 교육 목적 접근이 어렵다. Java OOP 설계 역량을 직접 적용할 수 없고 내부 로직 수정이 불가능하다.

나. 오픈소스 물류 시뮬레이터

1) 대표 사례

OpenTSim, ContainerSim (GitHub 기반 Python / C++ 프로젝트)

2) 기능적 특징

기본적인 컨테이너 흐름 시뮬레이션을 제공하며, 일부는 Python 기반으로 데이터 분석에 특화되어 있다.

3) 한계점

Java/JavaFX 기반이 아니어서 교과목 학습 목적에 부합하지 않는다. 실시간 시각적 모션 및 화주 UI가 부재하다.

다. Java 기반 학습용 시뮬레이터

1) 대표 사례

콘솔 기반 항만 관리 예제 프로그램 (교재 실습 수준)

2) 기능적 특징

선박·부두 등 핵심 객체를 OOP로 모델링하는 학습 예제를 제공한다.

3) 한계점

GUI가 없고 콘솔 출력 수준에 그친다. JavaFX 애니메이션, 멀티스레드 동시 처리, Gradle 빌드 구조가 적용되지 않는다.

2.2. 기존 솔루션과 본 프로젝트의 비교표

	Java OOP	JavaFX GUI	멀티 스 레드	크레인 모션	화주 UI	Gradle 빌드	맵 시각 화
본 프로젝트	○	○	○	○	○	○	○
SIMPORT	×	○	○	○	×	×	○
SimPy 기반	×	△	○	△	×	×	△
OpenTSim	×	×	△	×	×	×	×
Java 교재 예제	○	×	×	×	×	×	×

※ ○: 지원, △: 부분 지원, ×: 미지원

2.3. 차별성

가. 기존 솔루션

- 상용 도구는 높은 비용과 폐쇄 구조로 교육 목적 활용에 한계가 있다.
- 오픈소스는 Java/JavaFX 기반이 아니며 실시간 애니메이션 모션이 부재하다.
- 학습용 Java 예제는 GUI와 멀티스레드 없이 콘솔 수준에 그친다.

나. 본 프로젝트

- JavaFX(FXML + Controller) 아키텍처로 UI와 비즈니스 로직을 명확히 분리한다.
- Gradle(com.example) 빌드로 모듈 간 의존성을 체계적으로 관리한다.
- 화주(Shipper) 시점의 UI를 제공하여 실제 항만 이용자 관점에서 시스템을 체험할 수 있다.

3. 주요 기능

3.1. 하역장 맵 시각화 및 선박 이동 구현

- JavaFX Canvas 위에 항구, 부두(Berth), 대기 구역, 창고 구역을 2.5D로 시각화한다.
- AnimationTimer를 활용해 선박 객체가 목적 부두까지 부드럽게 이동하는 애니메이션을 구현한다.
- 목적국 표시: A안(깃발 이미지 오버레이) 또는 B안(나라별 특징 맵 배경 반영) 선택 구현.
- FIFO(LinkedBlockingQueue) 대기열로 다수 선박의 부두 배정 순서를 공정하게 제어한다.

3.2. 크레인 적재·하역 모션 및 화물 창고 구현

- JavaFX Timeline + KeyFrame으로 갠트리 크레인이 컨테이너를 집어 창고로 이동하는 역동적 모션을 구현한다.
- 하역 중인 선박의 진행률(%), 컨테이너 처리량, 효율 메트릭을 실시간으로 표시한다.
- 화물 창고(Warehouse) 객체의 구역별 재고 상태를 그리드 형태로 시각화한다.
- 멀티스레드(ExecutorService)로 여러 부두에서 동시에 적재·하역 작업이 병렬 진행된다.

3.3. 화주(Shipper) 시점 UI 구현

- JavaFX FXML 기반 입력 폼에서 화주가 화물 정보를 직접 입력한다.
- 입력 항목: 컨테이너 수, 목적국, 화물명/내용, 고유 식별 번호, 화물 유형 (약 5개 항목).
- Controller가 입력값을 Cargo 객체로 변환하여 시뮬레이터 엔진에 즉시 전달한다.
- Platform.runLater()를 활용해 JavaFX UI 스레드 안전성을 보장한다.

3.4. 멀티스레드 기반 동시성 제어

- ExecutorService(스레드 풀)로 다수 선박의 동시 운항 및 하역 작업을 처리한다.
- 부두(Berth)·크레인(Crane) 등 공유 자원 접근을 synchronized / ReentrantLock으로 제어한다.
- JavaFX의 Platform.runLater()로 백그라운드 스레드의 UI 갱신을 안전하게 처리한다.

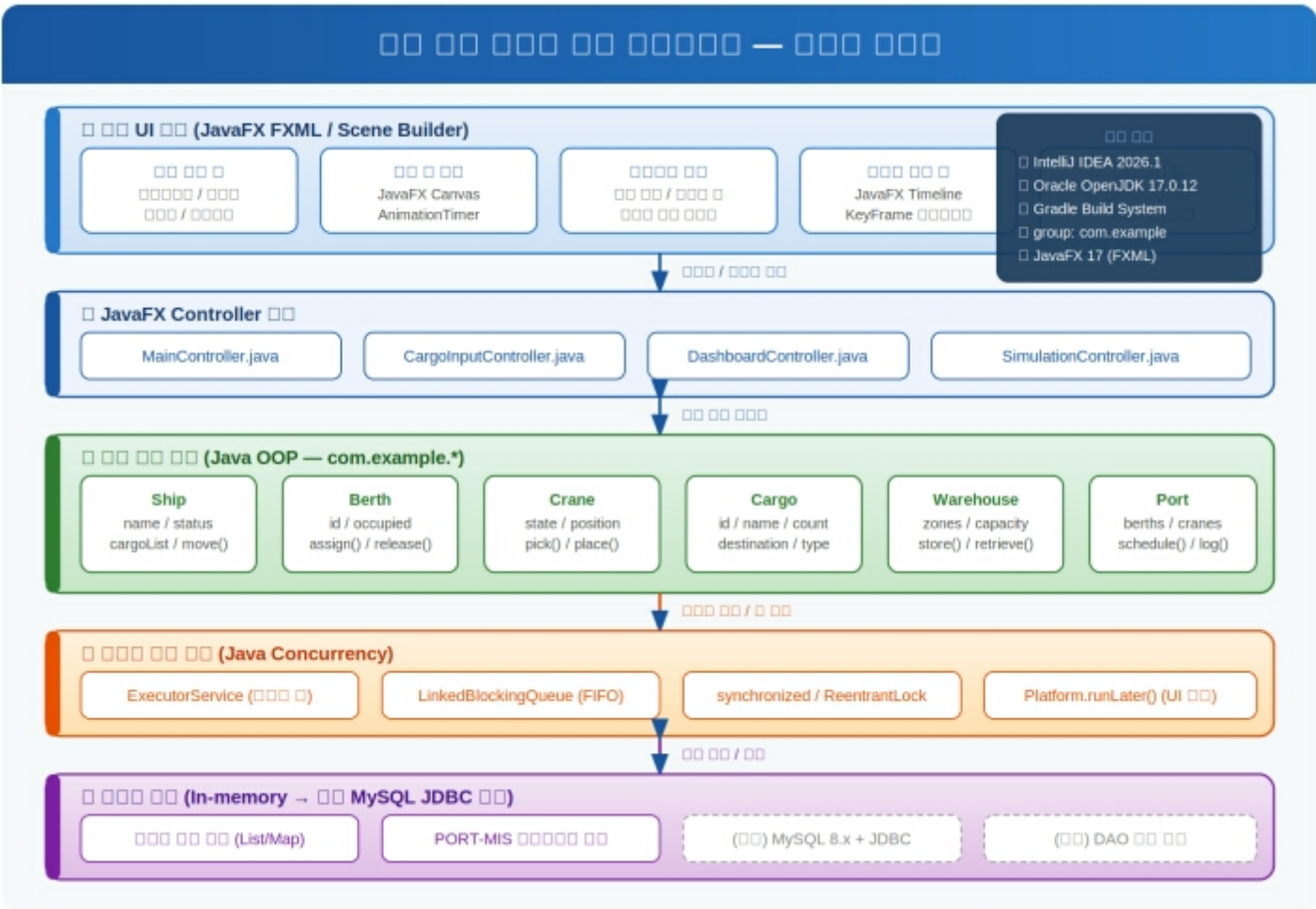
3.5. 가상 항만 대시보드

- 부두 점유 현황(OCCUPIED / AVAILABLE / BUSY / EMPTY), 활성 스레드 수를 실시간 모니터링한다.
- 선박 도착, 부두 배정, 하역 완료 등의 시스템 이벤트를 로그를 화면에 실시간 스트리밍한다.
- 전체 운영 효율성 메트릭(컨테이너 처리량, 평균 대기 시간 등)을 수치로 표시한다.

4. 시스템 구성도

4.1. 전체 시스템 아키텍처

본 시스템은 5개 계층으로 구성된 레이어드 아키텍처로 설계된다. JavaFX MVC 패턴을 중심으로 화주 UI 입력부터 핵심 OOP 처리 엔진, 동시성 제어, 데이터 관리까지 각 계층이 명확한 책임을 갖는다.



[그림 1] 항만 물류 디지털 트윈 시뮬레이터 시스템 구성도

4.2. 계층별 설명

계층	명칭	역할
①	화주 UI 계층	JavaFX FXML 기반 화물 입력 폼, 항만 맵 뷰어, 대시보드, 크레인 모션 뷰, 창고 현황 뷰 제공
②	Controller 계층	JavaFX Controller가 UI 이벤트를 수신하여 핵심 엔진으로 전달; UI-비즈니스 로직 분리

③	핵심 처리 엔진	Ship / Berth / Crane / Cargo / Warehouse / Port 객체의 OOP 모델링 및 비즈니스 로직 처리
④	동시성 제어 계층	ExecutorService(스레드 풀), LinkedBlockingQueue(FIFO), synchronized, Platform.runLater() 활용
⑤	데이터 계층	메모리 내 상태 관리(List/Map); 향후 MySQL JDBC 연동 및 PORT-MIS 공공 데이터 연계 예정

5. 핵심 구현 기술

5.1. 기반 언어 및 프레임워크

가. Java 17 (Oracle OpenJDK 17.0.12) + JavaFX 17

Java LTS 버전인 17을 채택하여 Record, Pattern Matching, Text Blocks 등 최신 문법을 활용한다. JavaFX 17은 Java와 라이프사이클을 공유하는 LTS 버전으로, FXML 기반 MVC 아키텍처와 AnimationTimer·Timeline 애니메이션 API를 통해 실시간 시각화를 구현한다.

나. Gradle Build System (group: com.example)

Gradle을 빌드 시스템으로 사용하여 JavaFX 플러그인 의존성 관리, 멀티 모듈 프로젝트 구조, 태스크 자동화를 수행한다. 핵심 build.gradle 구성은 다음과 같다.

```
// build.gradle (Project 수준)
plugins {
    id 'application'
    id 'org.openjfx.javafxplugin' version '0.1.0'
}
group = 'com.example'
version = '1.0.0'
sourceCompatibility = '17'
javafx {
    version = '17'
    modules = ['javafx.controls', 'javafx.fxml']
}
mainClassName = 'com.example.MainApp'
```

5.2. JavaFX 시각화 기술

기술 요소	적용 내용
JavaFX Canvas	2.5D 항만 맵, 선박, 크레인, 컨테이너를 픽셀 단위로 직접 그려 시각화
AnimationTimer	매 프레임(~60fps) 호출되는 handle()에서 선박 좌표를 갱신하여 이동 애니메이션 처리
Timeline + KeyFrame	크레인 집기·이동·내려놓기 동작을 KeyFrame 시퀀스로 정의하여 모션 구현
FXML + Scene Builder	화주 입력 폼, 대시보드 등 UI 레이아웃을 FXML로 선언; Controller와 분리
Platform.runLater()	백그라운드 스레드에서 발생한 상태 변화를 JavaFX UI 스레드에 안전하게 반영

ObservableList	창고 재고, 로그 리스트를 ObservableList로 관리하여 UI 자동 갱신
----------------	--

5.3. OOP 설계 원칙 적용

OOP 원칙	적용 클래스	구현 내용
캡슐화	Ship, Cargo, Berth	상태 변수를 private 선언; getter/setter로만 외부 접근 허용
상속	ShipType → ContainerShip, BulkShip	선박 유형별 화물 처리 로직을 상속으로 확장
다형성	Crane.operate(Ship s)	선박 유형에 따라 적재·하역 동작이 다형적으로 실행
인터페이스	Schedulable, Movable	스케줄링 가능 객체, 이동 가능 객체를 인터페이스로 추상화

5.4. 동시성 제어 기술

기술 요소	적용 내용
ExecutorService (스레드 풀)	Executors.newFixedThreadPool()로 선박별 스레드를 관리; 스레드 생성 비용 절감
LinkedBlockingQueue	부두 만석 시 선박을 FIFO 대기열에 삽입; 부두 반납 시 poll()로 순서 배정
synchronized 블록	Berth.assign() / Crane.pick() 등 공유 자원 접근 시 임계 구역 보호
ReentrantLock	창고(Warehouse) 입출고 작업 시 재진입 가능 락으로 세밀한 동시성 제어
Platform.runLater()	BackGround Thread → JavaFX UI Thread 안전 전환; UI 갱신 보장

6. 개발 환경 명세

6.1. 개발 환경 상세

항목	내용
IDE	IntelliJ IDEA 2026.1 (Community / Ultimate)
JDK	Oracle OpenJDK 17.0.12 (LTS)
UI 프레임워크	JavaFX 17 (FXML + Scene Builder)
빌드 시스템	Gradle 8.x (Kotlin DSL 또는 Groovy DSL)
프로젝트 group	com.example
메인 클래스	com.example.MainApp
버전 관리	Git / GitHub
운영체제	Windows 10/11 (개발 및 실행 환경)

6.2. 프로젝트 패키지 구조

```
com.example/  
├── MainApp.java           // JavaFX Application 진입점  
├── controller/  
│   ├── MainController.java  
│   ├── CargoInputController.java  
│   ├── DashboardController.java  
│   └── SimulationController.java  
├── model/  
│   ├── Ship.java  
│   ├── Berth.java  
│   ├── Crane.java  
│   ├── Cargo.java  
│   ├── Warehouse.java  
│   └── Port.java  
├── simulation/  
│   ├── SimulationEngine.java  
│   └── PortScheduler.java  
└── resources/fxml/  
    ├── main.fxml  
    ├── cargo_input.fxml  
    └── dashboard.fxml
```

7. 사용 데이터

7.1. 참조용 오픈 데이터

1) 설명:

해양수산부 공공데이터 포털 및 오픈소스 플랫폼에서 확보한 선박 입출항 정보, 컨테이너 처리량 등 항만 운영 관련 실제 데이터를 시뮬레이터의 파라미터 기준으로 참조한다.

2) 목적:

선박 수, 화물량, 처리 시간 등의 시뮬레이션 파라미터를 현실적인 값으로 설정하며, 향후 MySQL DB 연동 시 실제 데이터를 적재하여 시뮬레이션 현실성을 높인다.

7.2. 데이터 목록 및 링크

데이터 명	내용	출처	링크
PORT-MIS 선박 입출항	선박명, 입항 시각, 부두 번호, 화물 종류 등	해양수산부	공공데이터 포털
컨테이너 처리량 통계	연도별 항만별 컨테이너 처리 TEU 통계	해양수산부	해운항만물류정보시스템
선박 제원 정보	선박 유형, 총톤수, 길이, 화물 최대 적재량	한국선급(KR)	e-navigation
항만 부두 현황	전국 항만 부두 수, 길이, 접안 가능 선박 제원	해양수산부 항만 통계	항만물류정보
이시하라식 참조 (색각 시물 선택)	(선택적) UI 색상 접근성 검증용 참조 데이터	Kaggle	Kaggle Dataset

7.3. 향후 데이터 연동 계획

- 방학 중 MySQL 8.x + JDBC 연동을 통해 공공데이터를 실제 DB에 적재하여 시뮬레이션에 반영한다.
- 해양수산부 PORT-MIS REST API를 연결하면 실시간 선박 입항 데이터 기반 관제 시스템 초기 모델로 발전 가능하다.
- DAO 패턴(Data Access Object)을 적용하여 DB 연동 시 기존 비즈니스 로직 변경 없이 확장한다.

8. 개발 일정

[illegible]

9. 업무 분장

담당	역할	작업 상세 내용
이원훈	하역장 맵 및 선박 이동 구현	- JavaFX Canvas 기반 2.5D 항만 맵 레이아웃 설계 (항구·부두·대기구역·창고) - Ship 객체 OOP 모델링 및 AnimationTimer 기반 이동 애니메이션 구현 - 목적국 표시 구현: A안(깃발 이미지) / B안(나라별 배경 맵) - FIFO 대기열(LinkedBlockingQueue) 및 부두 배정 스케줄링 로직 구현
허은총	크레인 모션 구현 및 화물 창고 구현	- JavaFX Timeline + KeyFrame 기반 갠트리 크레인 적재·하역 모션 구현 - Crane 객체 상태 머신(IDLE→PICK→MOVE→PLACE) 설계 및 구현 - Warehouse 객체 구현: 구역별 재고 관리, 입출고 로직, 그리드 시각화 - 멀티스레드(ExecutorService)를 활용한 동시 하역 작업 병렬 처리
정승호	UI 구현 (화주 시점 인터페이스)	- JavaFX FXML 기반 화주 입력 폼 구현 (컨테이너 수·목적국·화물명·식별번호 등 5개 항목) - CargoInputController: 입력 데이터를 Cargo 객체로 변환하여 시뮬레이션 엔진에 전달 - 가상 항만 대시보드 FXML UI 구현 (부두 현황·스레드 수·시스템 로그·운영 메트릭) - Platform.runLater() 활용 UI 스레드 안전성 보장 및 전체 모듈 통합
공통	기획 및 품질관리	- Gradle 프로젝트 초기 설정 및 build.gradle 공동 관리 (group: com.example) - GitHub 저장소 관리, 코드 리뷰, 품질 관리 및 통합 테스트 - 문서 작업 및 최종 발표 자료 준비

참고문헌

- [1] Oracle, Java SE 17 Documentation, Oracle Corporation, <https://docs.oracle.com/en/java/javase/17/>
- [2] OpenJFX, JavaFX 17 Documentation, <https://openjfx.io/javadoc/17/>
- [3] Gradle, Gradle Build Tool Official Documentation, <https://docs.gradle.org>
- [4] openjfx, JavaFX Gradle Plugin, <https://github.com/openjfx/javafx-gradle-plugin>
- [5] 해양수산부, PORT-MIS 공공데이터 포털, <https://data.go.kr/data/15058820/openapi.do>
- [6] 해양수산부, 해운항만물류정보시스템(SP-IDC), <https://spidc.go.kr>
- [7] 한국선급(KR), e-navigation 선박 정보 포털, <https://www.e-navigation.kr>
- [8] B. Goetz et al., Java Concurrency in Practice, Addison-Wesley, 2006. (멀티스레드 설계 참조)
- [9] E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.